Closure at the Adjudication Seam

Universal recovery closure, certified abstention, and the necessary distinction in an adequate refinement

A complete mathematical treatment with formal proofs

24 October 2026 R8

Abstract

We prove a universal obstruction. If two admitted cases have the same readout and unequal target values, no decoder of that readout recovers the target on the admitted domain. The proof excludes the entire decoder class in one argument. Arbitrary processing of the unchanged readout, including complete adaptive transcripts, preserves the obstruction. We further prove that every globally sound attempted decoder must abstain at every stage on the certified ambiguous reading. The null case consequently has a positive certificate. Any adequate refinement must restore a distinction between the witnesses; that essential distinction cannot itself be decoded from the old readout.

The treatment gives the complete proofs, logical foundations, exact factorization result, observer-inclusive formulation, countermodels fixing the hypotheses, diagonal connection and worked Rule 30 instances. The central formal development contains 18 theorems and has no axiom dependencies reported by Lean. The accompanying packet supplies the full source corpus, isolated builds, axiom audits and retained antecedent proof packets.

Conceptual programme: Lu Semita / EmergenceByDesign / MetaLabe.

Mathematical exposition and formal continuation developed with ChatGPT.

Prior parallax development: Claude with Lu Semita.

Contents

1 The result and its mathematical standing **1**
1.1 The three conclusions established together . . . 1

2 Objects, readouts, targets and witness data **1**
2.1 Exact domain accounting . . . 2

3 Universal recovery closure: complete proof **2**
3.1 The class is fully quantified . . . 3
3.2 The exact logical derivation . . . 3

4 Processing, adaptive computation and unchanged information **3**

5 The null case has a positive certificate **4**

6 The necessary distinction and the recovery ladder **5**

7 Exact factorization and all boundary cases **6**

8 The observer-inclusive cut and the adjudication seam **7**
8.1 A completed metalevel conclusion . . . 7
8.2 The standard applies to its own application . . . 8

9 A complete finite model and its null case **8**

10 Diagonalization and the checked connection to provability **9**
10.1 The diagonal construction . . . 9
10.2 Quotation constructs the semantic fixed point . . . 9
10.3 Both proof and refutation status can be retained . . . 10

11 Rule 30: complete worked seams and dynamical bridges **10**
11.1 The white-driven right half-line . . . 10
11.2 Certificate entry or a proved settling residue . . . 11
11.3 The left trace: exact inversion and finite windows . . . 12
11.4 The actual ring and the finite-support seam . . . 12
11.5 The two-sided reduction and the remaining dynamical predicate . . . 13

12 Proof standing, class closure and the standard of challenge **14**
12.1 Why more unchecked candidates do not defeat this proof . . . 14
12.2 The common standing of mathematical closure results . . . 14
12.3 What is closed and what a new question changes . . . 15

13 Foundations and the complete axiom ledger **15**
13.1 The logical base of the central proof . . . 15
13.2 Extensions and their separate dependencies . . . 15
13.3 Parameters, hypotheses and admitted application predicates . . . 16

14 Verification, full sources and reproducibility **16**
14.1 Rebuild commands . . . 17
14.2 Integrity and verification limits . . . 17
14.3 The conclusion carried forward . . . 17

A Complete standalone core proof source **18**
A.1 UniversalRecoveryClosure.lean . . . 18
A.2 ClosureConsequences.lean . . . 18

B Complete core axiom audit **21**

C Proof source map **22**

D Named continuity record **23**

1 The result and its mathematical standing

The principal conclusion is exact:

$$O(a) = O(b), \quad T(a) \neq T(b) \quad \Longrightarrow \quad \neg\exists D\ \forall x \in X,\ D(O(x)) = T(x). \tag{1}$$

This is a theorem about a whole function class. It is established by a deduction, not by checking a large sample of candidate decoders. An untested decoder is already included in the quantified class. The existence of more candidates does not reopen the conclusion.

Throughout, *closure* means a proved resolution of a specified mathematical question. Here the question is whether the declared readout admits a decoder that is correct on the declared domain for the declared target. Equation (1) resolves that question negatively when its witness data are present. We call this *universal recovery closure*.

The *adjudication seam* names its observer-inclusive application. An adjudicator using only a readout is one of the functions covered by the theorem. The adjudicator's role does not exempt its procedure from the obstruction. If it requires a distinction that its reading erases, calling its processing "verification" does not recreate the missing distinction.

The theorem's force is the ordinary force of a proved mathematical implication: its stated hypotheses entail its stated conclusion. Its foundational status is explicit. The core proof uses equality, function application and contradiction in the form of negation introduction. It does not use excluded middle, choice, an assumption of an infinite search, or an assumption of a privileged external observer.

1.1 The three conclusions established together

1. **Whole-class exclusion.** Every total decoder of the old readout is excluded from global target recovery.
2. **Certified null case.** A stage-indexed attempt that is sound on every admitted case returns no target value on the ambiguous reading at any stage. This is a positive, universal proof of abstention.
3. **Necessary ladder step.** An adequate refinement must distinguish the certified witnesses. The essential added coordinate is not recoverable from the old readout.

Each conclusion receives a full proof below. The applications retain their own domains and targets, so that their conclusions are equally exact.

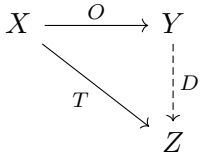
2 Objects, readouts, targets and witness data

Definition 2.1 (Recovery)**.** Let X, Y, Z be types, $O : X \to Y$ a readout and $T : X \to Z$ a target. Define

$$\mathrm{Recovers}(O, T) \ :\Longleftrightarrow\ \exists D : Y \to Z\ \forall x : X,\ D(O(x)) = T(x).$$

The admitted cases are precisely the elements of X. Correctness is exact equality in the specified target type Z.

A decoder makes the following triangle commute:



The recovery question asks whether such a D exists. It does not assume that O and T are identical functions or that a representation and its referent have been identified as an intrinsic essence.

Definition 2.2 (Seam certificate)**.** A seam certificate for (O, T) consists of

$$a, b : X, \qquad h_O : O(a) = O(b), \qquad h_T : T(a) \neq T(b).$$

These are the exact fields of the existing `SeamCertificate` structure: two witnesses, one equality of readings and one inequality of targets.

The seam is relative to a target. Information can be erased without harming a target that does not require it. Conversely, once the stated target needs the erased distinction, the obstruction is unavoidable. The hypotheses express that difference explicitly.

The formulation permits finite or infinite domains, numerical or symbolic targets, and mathematical objects representing persons, procedures, sentences or states. No binary ontology is imposed on X, Y or Z. Boolean examples below instantiate the theorem; they do not define its generality.

2.1 Exact domain accounting

If an application admits only cases satisfying $A(x)$, use the subtype $X_A = \{x : X \mid A(x)\}$. A certificate for that application must have both witnesses in X_A. The theorem therefore records admission as part of its data. Removing a witness by changing the domain changes the recovery question; it does not produce a decoder refuting the original theorem.

The existence of a and b already supplies elements of X and unequal elements of Z. There is no hidden non-emptiness assumption in the obstruction theorem. General converse results require separate attention to empty types and unused readings; Section 7 supplies it.

3 Universal recovery closure: complete proof

Theorem 3.1 (Two witnesses exclude the entire decoder class)**.** *For any X, Y, Z, O, T, a, b, if $O(a) = O(b)$ and $T(a) \neq T(b)$, then $\neg\,\mathrm{Recovers}(O, T)$.*

Proof. Assume recovery. Its witness is a function $D : Y \to Z$ with a proof $h(x) : D(O(x)) = T(x)$ for every $x : X$. Function congruence applied to the readout equality gives

$$D(O(a)) = D(O(b)).$$

Compose this equality with the correctness equalities at the two witnesses:

$$T(a) \;=\; D(O(a)) \;=\; D(O(b)) \;=\; T(b).$$

Applying $h_T : T(a) = T(b) \to \bot$ gives a contradiction. Thus every assumed recovery witness entails $\bot$, which is the definition of $\neg\,\mathrm{Recovers}(O, T)$. □

Formal receipt: `UniversalRecoveryClosure.no_decoder_recovers_erased_distinction`.

Proposition 3.2 (The same pair suffices for every decoder)**.** *For every $D : Y \to Z$, it is impossible that both $D(O(a)) = T(a)$ and $D(O(b)) = T(b)$ hold.*

Proof. The two asserted equalities, together with congruence of D under $O(a) = O(b)$, produce the same forbidden equality $T(a) = T(b)$. □

Formal receipt: `ClosureConsequences.two_cases_exclude_every_decoder`.

This is the exact meaning of a finite certificate for a universal obstruction. The certificate is not a finite sample offered as a substitute for an infinite proof. It is input to a proof that treats an *arbitrary* D. The pair does not depend on which decoder is proposed.

Constructively, Proposition 3.2 states that no decoder is correct on both witnesses. It does not need an additional choice of which of the two correctness statements fails. That stronger pointwise selection is unnecessary for global non-recovery.

3.1 The class is fully quantified

Define the adequate decoder class

$$\mathcal{D}_{O,T} = \{D : Y \to Z \mid \forall x : X,\ D(O(x)) = T(x)\}.$$

The theorem states $\mathcal{D}_{O,T} = \varnothing$. Every function of the declared type is covered, including ones never enumerated, never computed, or not computable. Thus the statement is stronger than the exclusion of some tested algorithms. Its proof has no remaining decoder case.

3.2 The exact logical derivation

In a context containing the typed data and h_O, h_T, the desired proposition is $(\exists D\ \forall x, D(O(x)) = T(x)) \to \bot$. The inference steps are:

1. Assume an element of the existential proposition and eliminate it, obtaining D and its correctness proof h.
2. Specialize h at a and b by universal elimination.
3. Apply equality congruence to h_O under D.
4. Reverse the equality at a and compose the three equalities.
5. Apply h_T to the resulting equality.

The assumption has yielded $\bot$, so abstraction over it proves negation. This is direct negation introduction. It does not appeal to the classical principle of deriving P from $\neg\neg P$.

4 Processing, adaptive computation and unchanged information

Theorem 4.1 (Arbitrary processing preserves the obstruction)**.** *Under the hypotheses of Theorem 3.1, for every type R, every $F : Y \to R$ and every $D : R \to Z$,*

$$\neg\big(\forall x : X,\ D(F(O(x))) = T(x)\big).$$

Proof. Congruence first gives $F(O(a)) = F(O(b))$ and then $D(F(O(a))) = D(F(O(b)))$. Correctness at both witnesses would imply $T(a) = T(b)$. Apply the target inequality. □

Formal receipt: `UniversalRecoveryClosure.no_processing_recovers_erased_distinction`.

Here R can itself be a function type. In particular, F can produce a complete sequence of intermediate states rather than a single finite summary. The theorem places no bound on the computational effort used to obtain F or D; it concerns what their inputs distinguish.

Definition 4.2 (Adaptive transcript)**.** For an initial-state function $I : Y \to S$ and an update function $A : Y \to S \to S$, define

$$\tau_y(0) = I(y), \qquad \tau_y(n+1) = A(y, \tau_y(n)).$$

The entire transcript is a function $\tau_y : \mathbb{N} \to S$.

Corollary 4.3 (Complete adaptive transcripts do not restore the distinction)**.** *For every* $D : (\mathbb{N} \to S) \to Z$*,*

$$\neg(\forall x,\ D(\tau_{O(x)}) = T(x)).$$

Proof. Apply Theorem 4.1 with $R = \mathbb{N} \to S$ and $F(y) = \tau_y$. Both the recursion and the final processing are covered by the same arbitrary-function theorem. □

Formal receipt: `ClosureConsequences.no_adaptive_transcript_decoder`.

An adaptive operation that obtains a new measurement of x is a different case: its complete reading must include that measurement. If the new reading splits the certified pair, the old obstruction no longer supplies a seam for that new reading. This is precisely the refinement analyzed next, not an untested decoder of the unchanged O.

5 The null case has a positive certificate

Definition 5.1 (Sound attempted decoder)**.** Let $Q : Y \to \mathbb{N} \to \mathrm{Option}(Z)$. The value None means that no target value is issued at that stage, while $\mathsf{Some}(z)$ reports z. Define soundness on the admitted domain by

$$\mathrm{Sound}(Q; O, T) \ :\Longleftrightarrow\ \forall x, n, z,\ Q(O(x), n) = \mathsf{Some}(z) \implies z = T(x).$$

This definition separates correctness from totality. It permits an attempt to abstain, and requires each value it does issue to be correct for every admitted case that could have supplied the reading.

Theorem 5.2 (Forced abstention on a certified seam)**.** *If* $O(a) = O(b)$*,* $T(a) \neq T(b)$ *and* Q *is sound, then*

$$\forall n \in \mathbb{N}, \quad Q(O(a), n) = \mathsf{None} \quad \textit{and} \quad Q(O(b), n) = \mathsf{None}.$$

Proof. Fix n. Inspect $Q(O(a), n)$. If it is None, the first equality is established. If it is $\mathsf{Some}(z)$, soundness at a yields $z = T(a)$. The equality $O(a) = O(b)$ gives $Q(O(b), n) = \mathsf{Some}(z)$, so soundness at b yields $z = T(b)$. Hence $T(a) = T(b)$, contradicting the certified inequality. The Some constructor is therefore impossible, leaving None.

Since n was arbitrary, the equality holds for every stage. Interchanging a and b proves the second statement. Case analysis on an option value is an inductive elimination rule and requires no excluded-middle axiom. □

Formal receipt: `ClosureConsequences.forced_abstention_at_seam`.

Formal receipt: `ClosureConsequences.both_witnesses_have_certified_abstention`.

Definition 5.3 (Certified abstention)**.**

$$\mathrm{CertifiedAbstention}(Q, y) \;:\Longleftrightarrow\; \forall n,\; Q(y, n) = \mathsf{None}.$$

Theorem 5.2 supplies such a certificate. Its conclusion is not "the procedure has not answered yet." It proves that every stage has the null value for this reading, provided the stated global soundness is retained. The null case is named and established by a uniform argument.

Corollary 5.4 (Sound total recovery is excluded)**.** *A sound Q cannot satisfy*

$$\forall x : X,\; \exists n, z,\; Q(O(x), n) = \mathsf{Some}(z).$$

Proof. Totality supplies such an n, z for a. Theorem 5.2 identifies the same output as None. The distinct constructors None and $\mathsf{Some}(z)$ cannot be equal. □

Formal receipt: `ClosureConsequences.no_sound_attempt_is_total`.

The model does not identify None with a negative verdict about $T(x)$. A negative verdict, when the target is Boolean, is a reported Boolean value and is subject to soundness like a positive one. Abstention is its own explicit output condition. The theorem applies equally to a proponent or a critic restricted to the same ambiguous reading.

If Q is implemented as bounded-stage simulation of a partial algorithm, the conclusion proves that no stage reports an answer on the shared reading. The abstract theorem does not require that implementation: it quantifies over every stage-indexed attempt of the specified type.

6 The necessary distinction and the recovery ladder

Theorem 6.1 (Every adequate added coordinate splits the seam)**.** *Let $C : X \to W$. If*

$$\mathrm{Recovers}(x \mapsto (O(x), C(x)), T),$$

then $C(a) \neq C(b)$.

Proof. Assume $C(a) = C(b)$. Together with $O(a) = O(b)$, product equality gives $(O(a), C(a)) = (O(b), C(b))$. A decoder of that joint reading must give the same result at the two witnesses, whereas global correctness requires $T(a)$ and $T(b)$. Their inequality gives a contradiction. □

Formal receipt: `ClosureConsequences.adequate_coordinate_must_distinguish`.

The same argument applies to an arbitrary adequate reading $V : X \to R$, whether or not it is initially written as an appended coordinate. If $V(a) = V(b)$, correctness of a decoder of V would imply $T(a) = T(b)$. Thus every adequate V splits the certified pair. Appending V to O retains the old reading as well, with no loss of that adequacy.

Formal receipt: `SeamIntegration.adequate_readout_splits_seam`.

Corollary 6.2 (The essential coordinate is not recoverable from the old readout)**.** *Under the same adequacy premise,* $\neg\,\mathrm{Recovers}(O, C)$.

Proof. Theorem 6.1 provides $C(a) \neq C(b)$, and the old reading still has $O(a) = O(b)$. Apply Theorem 3.1 to target C. □

Formal receipt: `ClosureConsequences.essential_coordinate_is_not_recoverable`.

This is the precise independence earned by the argument: the required coordinate does not factor through O. It is a functional non-recoverability claim, not an assertion of statistical independence, agency, or an additional physical spatial dimension.

Necessity does not by itself construct a sufficient coordinate. For example, on $X = \{0,1\}^2$, let O be constant, $C(x_1, x_2) = x_1$ and $T(x_1, x_2) = x_2$. The coordinate is not recoverable from O, and it splits the pair $(0,0), (1,1)$. Yet the refined reading still identifies $(0,0)$ and $(0,1)$, whose targets differ. The source corpus proves this exact four-state countermodel. It fixes the meaning of the necessary ladder step: an adequate refinement must split *every target-conflicting fibre*.

Formal receipt: `SeamClosureConditions.independent_coordinate_and_one_split_are_insufficient`.

A sufficient refinement is available whenever the target itself is retained: (O, T) recovers T by second projection. That proves formal adequacy of the joint representation. A practical method of obtaining the extra value is additional application data; the projection proof does not manufacture it.

Formal receipt: `ClosureConsequences.target_retention_recovers`.

7 Exact factorization and all boundary cases

Definition 7.1 (Constancy on readout fibres)**.**

$$\mathrm{FibreConstant}(O, T) \;:\Longleftrightarrow\; \forall a, b : X,\; O(a) = O(b) \implies T(a) = T(b).$$

Proposition 7.2 (Necessary fibre constancy)**.** $\mathrm{Recovers}(O, T)$ *implies* $\mathrm{FibreConstant}(O, T)$.

Proof. Take a decoder D. If $O(a) = O(b)$, correctness and congruence yield $T(a) = D(O(a)) = D(O(b)) = T(b)$. □

Formal receipt: `ClosureConsequences.recovery_requires_target_constancy`.

Theorem 7.3 (Exact image factorization)**.** *Let* $Y_O = \{y : Y \mid \exists x : X,\; O(x) = y\}$. *Let* $\widehat{O} : X \to Y_O$ *be the canonical image reading, taking* x *to* $O(x)$ *together with its image witness. There exists* $D_O : Y_O \to Z$ *with* $D_O(\widehat{O}(x)) = T(x)$ *for all* x *if and only if* $\mathrm{FibreConstant}(O, T)$.

Proof. The forward implication is the preceding proposition, using the image readout. For the reverse implication, choose for each $y \in Y_O$ a witness x_y with $O(x_y) = y$ and set $D_O(y) = T(x_y)$. For an actual reading $y = O(x)$, fibre constancy gives $T(x_y) = T(x)$, proving correctness. The value is independent of which witness is chosen because all witnesses of the same reading have the same target. □

Formal receipt: `SeamClosure.exact_image_recovery_iff`.

The displayed construction is the one formalized in the antecedent source. Its axiom audit reports `Classical.choice`. This dependency belongs to the converse construction. It is not a dependency of the obstruction, processing, abstention or necessary-refinement proofs.

Corollary 7.4 (Total factorization with a default)**.** *If a default* $z_0 : Z$ *is supplied, then*

$$\mathrm{Recovers}(O, T) \quad\Longleftrightarrow\quad \mathrm{FibreConstant}(O, T).$$

Proof. Use D_O on the image and return z_0 on unused readings. Every actual reading lies in the image, so the correctness proof is unchanged. The reverse implication is fibre constancy as above. □

Formal receipt: `SeamClosure.total_recovery_iff_with_default`.

The default handles a genuine edge case. If X is empty, Y is a singleton and Z is empty, fibre constancy is vacuous, but there is no total function $Y \to Z$. Image factorization still holds because the image is empty. No default is silently assumed. In a seam instance, the explicit values $T(a), T(b)$ already make Z inhabited, but the general converse must state its extension condition separately.

Two further cases complete the accounting. A constant target is recovered by a constant decoder, even if the readout discards many distinctions. An identity readout recovers every target by the target function itself. Thus the target-conflict hypothesis and the actual equality of readings are essential data, not ornamental qualifications.

Formal receipt: `ClosureConsequences.constant_target_is_recovered`.

Formal receipt: `ClosureConsequences.identity_readout_recovers`.

8 The observer-inclusive cut and the adjudication seam

The theorem applies to an adjudicator by recording its inputs. Let C be a type of criteria, observer positions or cuts, and let $V(c, x)$ be the target verdict under criterion c. Suppose an observation retains only $O(x)$. On the joint domain $C \times X$, define

$$\widetilde{O}(c, x) = O(x), \qquad \widetilde{T}(c, x) = V(c, x).$$

If $V(c, x) \neq V(d, x)$ for two criteria, then (c, x) and (d, x) form a seam: their retained observation is equal and their required verdict differs. Theorem 3.1 excludes every decoder of that erased-criterion readout.

Formal receipt: `SeamIntegration.erasing_cut_parameter_obstructs`.

If a calibrated decoder $D : C \to Y \to Z$ is supplied with $D(c, O(x)) = V(c, x)$, retaining $(c, O(x))$ recovers the target. The proof is direct substitution into the calibration equality.

Formal receipt: `SeamIntegration.retained_cut_parameter_recovers_with_calibration`.

This is the mathematical form of registering the cut in the account. In Lambda's terminology the target-relevant distinction is the self/other cut as expressed in the chosen representation. The formal statement records exactly when omission of that parameter prevents recovery and exactly what calibration suffices when it is retained. It grants no exemption to the person or procedure performing the adjudication.

8.1 A completed metalevel conclusion

The original target is $T(x)$. A different mathematical question asks whether the reading admits global recovery of T. A seam certificate settles this second question: its answer is no. With soundness, the same certificate settles the stage-indexed null case in Theorem 5.2: no target value is issued at any stage on the shared reading.

Thus recognition of the seam is a positive, completed conclusion about adequacy. It does not require first recovering the original target. The statement "this readout cannot recover this

target" is fully established even though the reading does not supply $T(a)$ in isolation from the witness data. These conclusions address different targets and coexist without contradiction.

The named *adjudication seam* preserves that distinction on return. One does not reset an established universal obstruction to "perhaps an untested decoder will work" within the unchanged class. A challenge to an application must address its witness data, admission conditions, logical derivation or correspondence to the intended claim. Repeating an excluded recovery attempt is not such a challenge.

8.2 The standard applies to its own application

Let $k : J \to K$ assign declared criteria to judgments and $v : J \to V$ assign verdicts. Say the verdict follows the declared standard if

$$\exists f : K \to V \; \forall j : J, \; v(j) = f(k(j)).$$

Then equal declared criteria require equal verdicts: if $k(a) = k(b)$,

$$v(a) = f(k(a)) = f(k(b)) = v(b).$$

This is a checked theorem with no axiom dependencies. It applies equally to positive claims and objections. The declaration that a standard is being followed is an explicit premise; consistency of its application does not by itself prove that its criteria are adequate for another target.

Formal receipt: `SeamClosure.equal_declared_criteria_require_equal_verdicts`.

A registration rule can additionally require each accepted certificate to carry its source and criterion. The antecedent development proves that a record with both fields absent cannot meet that rule. The rule is recorded as a hypothesis, and registration alone supplies no automatic truth verdict. This keeps the adjudicator inside the declared account without equating a record of participation with a proof of the target.

Formal receipt: `SeamClosureConditions.nullself_cannot_supply_registered_certification`.

9 A complete finite model and its null case

Let $X = Z = \{\mathsf{false}, \mathsf{true}\}$, $Y = \{*\}$, $O(x) = *$ and $T(x) = x$. The two cases give

$$O(\mathsf{false}) = O(\mathsf{true}), \qquad T(\mathsf{false}) \neq T(\mathsf{true}).$$

Consequently no function $D : Y \to Z$ recovers T. The entire two-element decoder class consists of the two constant Boolean functions; the same proof excludes both without enumerating them.

Formal receipt: `ClosureConsequences.bit_seam_excludes_every_decoder`.

Define $Q(*, n) = \mathsf{None}$ at every stage. It is sound: the premise that it returns $\mathsf{Some}(z)$ is impossible, so every implication in the soundness definition holds. Applying Theorem 5.2 gives a certificate $\forall n, Q(*, n) = \mathsf{None}$. The program's abstention is formally recognizable; there is no theorem here saying that all recursions are incapable of recognizing their own behavior.

Formal receipt: `ClosureConsequences.null_attempt_is_sound`.

Formal receipt: `ClosureConsequences.bit_null_has_a_certificate`.

Now retain the coordinate $C(x) = x$. The refined reading $(*, x)$ recovers T by second projection. It splits the certified pair and cannot be decoded from the old singleton reading. Every part of the obstruction, null case and sufficient refinement is instantiated in this finite model.

Formal receipt: `ClosureConsequences.bit_distinction_restores_recovery`.

For contrast within the same formal definitions, retain the constant readout but change the target to a fixed Boolean. A constant decoder now recovers it. The target has changed, so this is not a counterexample to the theorem with target $T(x) = x$. It demonstrates why a mathematical claim must retain both its observation map and its target when transported.

10 Diagonalization and the checked connection to provability

10.1 The diagonal construction

Lawvere's framework unifies several diagonal arguments [1]. The following set/function form is fully proved in the source corpus. Let $E : A \times A \to B$ and $v : B \to B$. Define

$$\Delta(x) = v(E(x, x)).$$

If $v(b) \neq b$ for every b, no row $E(c, -)$ equals Δ everywhere: at $x = c$ that equality would imply $E(c, c) = v(E(c, c))$. Conversely, if every function $A \to B$ is a row, apply that coverage to Δ. The resulting $E(c, c)$ is a fixed point of v.

Formal receipt: `DiagonalSeamBridge.diagonal_is_not_a_represented_row`.

Formal receipt: `DiagonalSeamBridge.point_surjection_forces_fixed_point`.

Suppose the rows represent every decoder after a given $O : A \to Y$:

$$\forall D : Y \to B \; \exists c \; \forall x, \; E(c, x) = D(O(x)).$$

If D recovered Δ, its row would equal Δ everywhere, contradicting the preceding construction. Therefore $\neg\,\mathrm{Recovers}(O, \Delta)$. This is the checked diagonal-to-recovery bridge.

Formal receipt: `DiagonalSeamBridge.represented_decoders_cannot_recover_diagonal`.

10.2 Quotation constructs the semantic fixed point

The R6 interface supplies sentence, unary-formula and code types, together with sentence quotation q_S, unary quotation q_U, instantiation I, sentence interpretation H, unary interpretation M, a diagonal code operation d, sentence provability P and coded provability P_c. Its four receipts are

$$\begin{aligned} H(I(u, c)) &\leftrightarrow M(u, c), \\ d(q_U(u)) &= q_S(I(u, q_U(u))), \\ P_c(q_S(s)) &\leftrightarrow P(s), \\ M(A, c) &\leftrightarrow \neg P_c(d(c)). \end{aligned}$$

Set $G = I(A, q_U(A))$. Substituting the receipts in order proves

$$H(G) \leftrightarrow \neg P(G).$$

The sentence is an explicit construction from the interface, not an assumed conclusion. Coding and provability are likewise explicit objects in mechanized incompleteness developments [2, 3].

Formal receipt: `DiagonalSeamBridge.constructed_sentence_has_diagonal_meaning`.

If local soundness $P(G) \to H(G)$ holds, a proof of G would yield both $P(G)$ and $\neg P(G)$. Hence $\neg P(G)$, and the equivalence gives $H(G)$. Together with an unprovable falsum whose

interpretation is false, these witnesses have equal provability status and unequal interpretations. The R6 construction packages them as the existing seam type.

Formal receipt: `DiagonalSeamBridge.seamFromDiagonalInterpretation`.

10.3 Both proof and refutation status can be retained

Let N be formal negation and let $O_{\pm}(s) = (P(s), P(Ns))$. Supply the stated receipts

$$H(NG) \leftrightarrow \neg H(G), \quad P(NG) \to H(NG), \quad P(NNG) \to P(G).$$

Since $H(G)$, soundness excludes $P(NG)$. Double-negation elimination and $\neg P(G)$ exclude $P(NNG)$. Thus

$$O_{\pm}(G) = O_{\pm}(NG) = (\mathsf{False}, \mathsf{False}), \qquad H(G) \neq H(NG).$$

The existing closure theorem applies directly, and every adequate added coordinate must distinguish G from NG.

Formal receipt: `DiagonalSeamBridge.godelSignedProofSeam`.

Formal receipt: `DiagonalSeamBridge.quotation_to_signed_recovery_obstruction`.

Formal receipt: `DiagonalSeamBridge.adequate_signed_refinement_must_split`.

Here H is the declared interpretation and P is actual derivability, not a finite failure to find a proof. The interface theorem proves the displayed implication from its supplied receipts. Instantiating it to a named arithmetic theory is the task of constructing those receipts for that theory. The abstract signature alone does not assert arithmetic expressiveness or prove independence of a particular Rule 30 predicate. These are exact differences between the theorem statements.

11 Rule 30: complete worked seams and dynamical bridges

The application packages retain the exact Rule 30 update, the single-black seed definition of `center30`, and the driven half-line definitions. They contain the complete formal dependency chains. This section proves the central application constructions and records the precise reduction statements that connect them to the center-column question.

11.1 The white-driven right half-line

Hold column 0 white. For $m \geq 1$ the right half-line evolves by

$$x_m(t+1) = x_{m-1}(t) \oplus (x_m(t) \vee x_{m+1}(t)).$$

If every positive initial cell is white, induction on time shows that every positive cell remains white. Otherwise a finite Boolean scan of a finitely supported start finds a first positive black cell at n. The zero cells preceding its leading front remain zero until the front reaches them, and the cell immediately to its left becomes black at the next step. The front therefore reaches column 1 at time $n - 1$.

Formal receipt: `Rule30WhiteBoundary.finite_start_dichotomy`.

Formal receipt: `Rule30WhiteBoundary.first_black_front`.

At column 1, the white boundary gives

$$x_1(t+1) = x_1(t) \vee x_2(t).$$

Once black, column 1 remains black regardless of the future edge value. This is a width-one, period-one robust certificate. The finite starts are therefore completely classified: the all-white positive half-line stays white; a start with a first black cell eventually has a permanently black first column.

Formal receipt: `Rule30WhiteBoundary.blackShield`.

Formal receipt: `Rule30WhiteBoundary.exact_finite_start_trace_dichotomy`.

For every width $J > 0$, compare the all-white start with a single black cell at $J + 1$. Their initial J-cell bands agree, while their column-1 values at time J are 0 and 1. Their times have the same phase modulo any period $P > 0$. Thus no phase-deterministic, free-edge certificate can contain the all-white band. The white orbit has the same band at every time, so it never enters any such finite certificate. Its whole trajectory is nevertheless described and proved inside the model.

Formal receipt: `Rule30WhiteBoundary.no_white_zero_band_certificate`.

Formal receipt: `Rule30WhiteBoundary.finite_nonentry_iff_white_residue`.

The same two starts form a seam for recovery of the entire column-1 trace from any fixed initial prefix. This is the R5 concrete temporal ladder. It is a statement across finite starts, not a complexity lower bound for computing one fixed-time cell of one known initial state.

Formal receipt: `Rule30WhiteBoundary.actualPrefixSeam`.

Formal receipt: `Rule30WhiteBoundary.no_fixed_prefix_recovers_all_finite_start_traces`.

11.2 Certificate entry or a proved settling residue

Let w be a nonempty periodic drive word. The verified bridge requires an explicit predicate R and proofs that every finite start either enters a shield or belongs to R, and that every finite start in R has eventually periodic first-column output. Shield entry supplies periodicity on the other branch. Hence every finite driven start settles.

A restart of the right half of the actual single-black-seed automaton has finite support by its light cone. If its center were eventually w, the restart would be driven by w. Applying finite-start settling gives the right-column periodicity used in the previously verified adjacent-column reduction. That reduction excludes the proposed center repetition. The complete reduction proof is in the imported application chain.

Formal receipt: `CertificateResidueBridge.settles_of_certificate_or_residue`.

Formal receipt: `CertificateResidueBridge.right_settles_of_finite_settling`.

Formal receipt: `CertificateResidueBridge.center_avoids_of_certificate_or_residue`.

For the white word, the residue is exactly the all-white positive start, and its settling proof is the induction already given. The bridge then excludes eventual repetition of the white word in the actual center. Universal shield entry is false in this worked case, while the weaker settling premise is true. The residue receives a positive dynamical proof.

Formal receipt: `Rule30WhiteBoundary.center_avoids_white_via_residue`.

Formal receipt: `Rule30WhiteBoundary.settling_is_strictly_weaker_than_shield_entry`.

11.3 The left trace: exact inversion and finite windows

For the left half-line, the positive index m denotes location $-m$ and

$$L_m(t+1) = L_{m+1}(t) \oplus (L_m(t) \vee L_{m-1}(t)).$$

If two starts agree before position $t+m$, exclusive-or cancellation gives

$$L_m^s(t) \oplus L_m^{s'}(t) = s(t+m) \oplus s'(t+m).$$

An induction on time proves this identity. At the induction step the lower-position terms agree by the light cone and cancel, leaving the outer start bit. Thus the bit at initial position $J+1$ first distinguishes the column--1 trace at time J under every drive.

Formal receipt: `LeftSeam.lhs_xor_entry`.

Formal receipt: `LeftSeam.trace_first_difference`.

The reconstruction reads one trace value at a time. After m steps it has reconstructed cells 1 through m; at the next step it exclusive-ors the prescribed trace value with the contribution of those existing cells. The exclusive-or entry identity proves that this is the next initial bit. Consequently the whole left trace recovers the start, and every trace is realized by the start decoded from it.

Formal receipt: `LeftSeam.rebuild_eq_trunc`.

Formal receipt: `LeftSeam.left_trace_recovers_start`.

Formal receipt: `LeftSeam.left_trace_onto`.

For a desired trace q, let $s_K = \mathrm{rebuild}(b, q, K)$. The reconstruction is supported within K. Inside the cone of $(t, 1)$ for $t < K$, it agrees with the full decoded start, so

$$\forall t < K, \quad L_1^{s_K}(t) = q(t).$$

This proves the finite-shadow theorem. Padding an initial prefix and reading its first K trace values gives the forward map; finite reconstruction gives the inverse. Stability of reconstructed cells and the cone prove both inverse identities. The first K start bits and first K trace bits are therefore in bijection.

Formal receipt: `FiniteWindowSeam.shadow_matches_trace`.

Formal receipt: `FiniteWindowSeam.forward_backward`.

Formal receipt: `FiniteWindowSeam.backward_forward`.

At a black drive time $t < K$, toggle initial cell $t+1$. This is an involution preserving support and every earlier pin test, while reversing the current test. It pairs the earlier survivors with exactly one survivor of the current pin in each pair. For $T \le K$, the ordinary finite-counting corollary is

$$\#\,\mathrm{Survivors}(K, T) = 2^{K - B(T)},$$

where starts are counted by their K positive initial bits and $B(T)$ counts black drive cells before T. The inverse maps and pairing are kernel-checked; this cardinality formula is their stated finite-counting corollary rather than a separate Lean cardinality declaration.

Formal receipt: `FiniteWindowSeam.black_pin_pairs_prior_survivors`.

11.4 The actual ring and the finite-support seam

The parallax package proves a seven-cell Rule 30 ring orbit of period four. Its columns are precisely the seven-column loop block used in the right certificate construction. Read leftward

from its black center, its initial half-line s_∞ has black cells at arbitrarily large indices and is not finitely supported. Let s_T be its truncation after positive cell T. Then s_T is finite and the first T start cells agree with s_∞. The cone gives equality of the first T left-trace values under every common drive b.

Formal receipt: `RingComparator.loop_block_is_the_ring_orbit`.

Formal receipt: `RingComparator.ringStart_not_finite`.

On the domain of all half-lines, write $\mathrm{FiniteSupport}(s) \leftrightarrow \exists K\ \forall i > K,\ s(i) = 0$ and let $W_{b,T}(s)$ be that finite trace window. Then

$$W_{b,T}(s_T) = W_{b,T}(s_\infty), \qquad \mathrm{FiniteSupport}(s_T), \qquad \neg\,\mathrm{FiniteSupport}(s_\infty).$$

These are the complete fields of a seam certificate. Hence

$$\forall b, T, \quad \neg\,\mathrm{Recovers}(W_{b,T}, \mathrm{FiniteSupport}).$$

There is no unexamined decoder left in this statement for any fixed window. The proof covers every T by the explicit family of witnesses.

Formal receipt: `FiniteWindowSeam.ringSupportWindowSeam`.

Formal receipt: `FiniteWindowSeam.no_finite_trace_window_recovers_support`.

Each fixed positive coordinate of s_T eventually equals the ring coordinate as T grows. Every truncation is finite, and the limiting start is not. This is an explicit finite-approximation result, proved without invoking a topology or compactness theorem.

Formal receipt: `FiniteWindowSeam.finite_truncations_converge_on_positive_cells`.

On the restricted domain of finite starts, the target FiniteSupport is given and constant. A constant true decoder recovers that target there. This does not contradict the theorem on the enlarged domain. It identifies exactly what the original finite-support axiom supplies and why the actual dynamical question must use it, rather than try to infer it from a fixed observation window.

11.5 The two-sided reduction and the remaining dynamical predicate

The actual center obeys

$$c(t+1) = L(t) \oplus (c(t) \vee R(t)).$$

For a periodic drive w, define a joint break as failure of this equality for its two driven half-lines. The checked two-sided premise is

$$\forall s_L, s_R \text{ finite}, \quad \exists t\ \ \mathrm{JointBreaksAt}(w, s_L, s_R, t).$$

If the actual center eventually repeated w, restart both halves at the onset. Their support is finite, their drive is w, and the actual center rule keeps holding. The premise supplies a violating time, a contradiction. This proves the implication from the premise to exclusion of w.

Formal receipt: `JointBreak.center_avoids_of_joint_breaks`.

For every finite horizon T, a different explicit construction supplies a finite pair keeping the center rule before T. Choose the right start white and prescribe the left trace

$$q(t) = c(t+1) \oplus (c(t) \vee R(t)).$$

The finite-shadow theorem realizes this trace up to T with support at most T. Exclusive-or cancellation verifies the center rule. Therefore

$$\forall T\ \exists s_L, s_R \text{ finite}\ \forall t < T,\ \neg\,\mathrm{JointBreaksAt}(w, s_L, s_R, t).$$

The pair depends on T. It is an arbitrary finite pair in the driven model, not an asserted restart of the single-black-seed orbit.

Formal receipt: `FiniteWindowSeam.every_finite_window_has_a_finite_pair`.

This excludes a width-independent universal break-time cap. It is compatible with every *fixed* finite pair eventually breaking. The one-sided version also proves that any valid width-dependent bound g satisfies $g(K) \geq K$. These quantified statements identify the frontier of the initial-bit argument: from time K onward there is no further freely chosen initial cell beyond the support edge.

Formal receipt: `FiniteWindowSeam.no_width_independent_joint_break_bound`.

Formal receipt: `FiniteWindowSeam.every_left_bound_dominates_width`.

The parallax predicate `AdmittedPattern` is the explicit one-sided all-width bound, equivalent in Lean to a universal family of finite Boolean checks. Its finite checks and its implication to the center conclusion are separate proved statements. An all-width proof of that predicate, or of the weaker joint premise, is not supplied by universal recovery closure. The certificate of a readout obstruction and the dynamical all-start predicate have different signatures.

Formal receipt: `LeftCheck.admitted_pattern_is_pi1`.

Formal receipt: `LeftCheck.problem1_from_admitted_pattern`.

12 Proof standing, class closure and the standard of challenge

12.1 Why more unchecked candidates do not defeat this proof

The objection "there may be another unchecked candidate" addresses a finite search claim only when that search leaves the candidate outside its coverage. Theorem 3.1 leaves no decoder of the stated type outside its quantifier. Proposing another such D repeats the arbitrary case already treated. Theorem 4.1 likewise includes every processing function of the old reading.

The same principle applies to the null case. A longer attempt does not escape Theorem 5.2: every stage is already quantified. The proof is a certificate about the procedure's whole stage sequence on the shared input. It does not extrapolate from the number of observed None outputs.

An actual counterexample to the application must meet its admission and typing conditions and challenge the witness equality, the target inequality, the soundness premise where used, or a step in the inference. Changing the readout, target or domain can define a successful new recovery problem. It does not refute the old theorem. The criterion is mathematical and applies equally to the original proponent and its critic.

12.2 The common standing of mathematical closure results

Lindemann's transcendence result excludes every nonzero rational polynomial vanishing at π; calculating further digits is not its proof [4]. The recovery theorem excludes every globally adequate decoder from the specified readout when a seam is present. Each is a universal mathematical conclusion established by an argument over its class, not by exhausting a list of examples.

Gödel's results concern formal provability with their own logical hypotheses [3]. The R6 construction gives a precise route from a semantic diagonal interface to the recovery seam. Turing's work supplies limits on general decision procedures [5]. Those results do not say that

no particular non-answering procedure can be recognized as such; Section 5 gives an explicit positive certificate in the present model. Their common mathematical standing does not erase the differences between their quantified objects and hypotheses.

The conclusion here is asserted with its full deductive force. A hypothesis is part of a theorem's meaning, not a reduction of its status to a guess. Once the seam data are supplied, non-recovery is settled for the entire declared class. The formal applications below supply those data in their stated domains.

12.3 What is closed and what a new question changes

The completed recovery question is

$$\exists D : Y \to Z \; \forall x : X, \; D(O(x)) = T(x).$$

The completed answer under a seam is its negation. The completed null question, with global soundness, is the stage-universal abstention certificate. A question about the target value in a new domain, an enriched reading, a specific dynamical invariant or a named theory's provability predicate is a different mathematical statement.

This preserves higher-level closure as an earned result. It also keeps the same standard for further claims: limited capacity to adjudicate an entire programme does not invalidate a correct local argument, and absence of a certificate does not become a universal refutation. Neither side receives a special exemption from the stated inference rules.

13 Foundations and the complete axiom ledger

13.1 The logical base of the central proof

The core is expressed in Lean 4.22.0. Types, functions, universal and existential quantification, equality, conjunction, negation and falsehood have their standard dependent-type-theoretic meanings. In particular, $\neg P$ abbreviates $P \to \text{False}$ and inequality abbreviates negated equality. The proof uses the eliminators for these types and transitivity, symmetry and congruence of equality.

Lean's transitive axiom audit reports *no axiom dependencies* for all 25 named declarations in the two-file core, including all 18 theorems. The core does not call `Classical.choice`, `propext`, `Quot.sound` or `sorryAx`. This is a precise statement about the checked proof terms. It does not mean that a proof has no logical foundation: the kernel's formation, elimination and computation rules are the declared foundation in which the proof is checked.

13.2 Extensions and their separate dependencies

The following ledger is obtained from fresh `#printaxioms` outputs, not inferred from a general label attached to a file.

Statement family	Reported axiom dependencies
All R8 core declarations, including processing, adaptive transcripts, forced abstention and required coordinate	None
Recovery implies fibre constancy; the original collision and processing theorems	None

Statement family	Reported axiom dependencies
Image recovery iff fibre constancy	`Classical.choice`
Total recovery iff fibre constancy, with default	`propext`, `Classical.choice`, `Quot.sound`
Added coordinate must split; essential coordinate not recoverable; erased criterion obstructs; calibrated criterion recovers	None
Equal declared criteria require equal verdicts	None
Four-state insufficiency and frozen-entry countermodels; registered nullself exclusion under its registration premise	None
Audited R6 diagonal application declarations	At most `propext` and `Quot.sound`
Audited R7 finite-window application declarations	At most `propext` and `Quot.sound`

The converse factorization result therefore has a different dependency ledger from the necessary obstruction. The ledger preserves that difference. No axiom needed by an extension is silently attributed to the core, and no extension is described as choice-free when its proof uses choice.

13.3 Parameters, hypotheses and admitted application predicates

A typed argument such as $h_O : O(a) = O(b)$ is a theorem hypothesis. A structure such as the diagonal interface supplies operations and receipts as parameters. A proof of a conditional implication does not also construct every possible application's parameters. Conversely, an actual worked seam whose witness equalities and inequalities are proved has discharged those hypotheses for that application.

For the Rule 30 finite-window and ring seams, the source constructs the witnesses and proves their properties. For the semantic diagonal bridge, the interface and local soundness receipts are explicit inputs. For the all-width breaking bound, the universal finite-check predicate is an explicit unproved premise of its center-pattern reduction. These entries state the exact propositions in the corpus and make their foundations inspectable.

14 Verification, full sources and reproducibility

The proof packet is a complete compilation rather than a replacement of the earlier versions. It contains the new standalone core, the full R6 and R7 source trees, the unmodified R6 and R7 packets, the saved adjudication checkpoint, the manuscript source and fresh verification receipts. The R6 packet retains its earlier R5, R4 and prior source packets; the R7 packet retains the supplied parallax report and supplement.

Fresh target	Module builds	Audited theorems	Named declarations
Standalone core	2	18	25
Diagonal R6 chain	19	24	34
Parallax R7 chain	22	22	33
Selected foundational statements	–	13	13
Total	43	77	105

The 43 module builds are isolated compilations of three targets, with shared antecedent modules rebuilt in each applicable target. They are not 43 distinct mathematical theories. Every stage completed at trust level 0 with zero compiler errors and warnings. The named audits record the checked declarations and their transitive dependencies.

14.1 Rebuild commands

Install the official Lean 4.22.0 release. From the extracted packet run:

```
python3 reproduction/verify_all.py --lean /absolute/path/to/lean
python3 reproduction/audit_foundations.py --lean /absolute/path/to/lean
```

Only bundled Lean libraries are needed. The two application chains use separate `LEAN_PATH` build directories, preserving their original module versions. Compiled `.olean` files are excluded from the distribution so that the proof sources are rebuilt.

The original restricted runtime required an optional executable-discovery shim. Its source is retained; it redirects only the process's own `/proc/PID/exe` lookup to `/proc/self/exe`. Lean's executable and kernel were unchanged. On a normal installation this shim is not needed. The optional flag `--compat-shim` is documented in the packet README.

14.2 Integrity and verification limits

The receipts include source hashes, version data, raw compiler output, exact signatures and axiom reports. A packet manifest hashes every distributed payload. The historical large C scans remain labeled as computational evidence in their original records; they were not rerun as part of this compilation. The complete formal chains and the stated axiom audits were rerun. The R7 packet also preserves the independently reproduced parallax Python checks and audit logs.

14.3 The conclusion carried forward

The central class-wide non-recovery theorem is proved. Its invariance under arbitrary processing is proved. The globally sound stage-indexed null case has a positive certificate. The necessary distinguishing coordinate in any adequate refinement is proved. The Rule 30 examples instantiate these structures with explicit states and complete checked sources. These results are retained as established mathematical statements with their exact quantified coverage.

A Complete standalone core proof source

These are the full two files of the standalone core. Their machine-readable versions are supplied unchanged beside the manuscript. The first file is the earlier self-contained universal closure statement; the second derives its processing, null-case and refinement consequences.

A.1 UniversalRecoveryClosure.lean

```
import Lean

/-!
Universal recovery closure, 24 October 2026.
A self-contained statement of the collision argument already used in the
seam work. The proof excludes the entire declared decoder class at once.
It uses no custom assumptions beyond its explicit theorem hypotheses.
-/

universe u v w r

namespace UniversalRecoveryClosure

theorem no_processing_recovers_erased_distinction
    {X : Type u} {Y : Type v} {Z : Type w}
    (observe : X → Y) (target : X → Z) (a b : X)
    (sameReading : observe a = observe b)
    (differentTargets : target a ≠ target b) :
    ∀ {R : Type r} (process : Y → R) (decode : R → Z),
      ¬ (∀ x, decode (process (observe x)) = target x) := by
  intro R process decode correct
  apply differentTargets
  calc
    target a = decode (process (observe a)) := (correct a).symm
    _ = decode (process (observe b)) := congrArg (fun y => decode (process y)) sameReading
    _ = target b := correct b

theorem no_decoder_recovers_erased_distinction
    {X : Type u} {Y : Type v} {Z : Type w}
    (observe : X → Y) (target : X → Z) (a b : X)
    (sameReading : observe a = observe b)
    (differentTargets : target a ≠ target b) :
    ¬ (∃ decode : Y → Z, ∀ x, decode (observe x) = target x) := by
  intro recovery
  obtain ⟨decode, correct⟩ := recovery
  exact no_processing_recovers_erased_distinction observe target a b
    sameReading differentTargets (fun y => y) decode correct

#print axioms no_processing_recovers_erased_distinction
#print axioms no_decoder_recovers_erased_distinction

end UniversalRecoveryClosure
```

A.2 ClosureConsequences.lean

```
import UniversalRecoveryClosure

/-!
Consequences of universal recovery closure, R8, 24 October 2026.
The null case is proved from a seam and global soundness, rather than inferred
from elapsed search. Every declaration below has an explicit axiom audit.
-/

namespace ClosureConsequences

open UniversalRecoveryClosure

universe u v w r s
```

```
variable {X : Type u} {Y : Type v} {Z : Type w}

def Recovers (observe : X → Y) (target : X → Z) : Prop :=
  ∃ decode : Y → Z, ∀ x, decode (observe x) = target x

theorem two_cases_exclude_every_decoder
    (observe : X → Y) (target : X → Z) (a b : X)
    (same : observe a = observe b) (different : target a ≠ target b)
    (decode : Y → Z) :
    ¬ (decode (observe a) = target a ∧ decode (observe b) = target b) := by
  intro both
  apply different
  exact both.1.symm.trans ((congrArg decode same).trans both.2)

theorem universal_class_is_closed
    (observe : X → Y) (target : X → Z) (a b : X)
    (same : observe a = observe b) (different : target a ≠ target b) :
    ¬ Recovers observe target :=
  no_decoder_recovers_erased_distinction observe target a b same different

def adaptiveTrace {State : Type r} (initial : Y → State) (advance : Y → State → State)
    (reading : Y) : Nat → State
  | 0 => initial reading
  | n + 1 => advance reading (adaptiveTrace initial advance reading n)

theorem no_adaptive_transcript_decoder
    (observe : X → Y) (target : X → Z) (a b : X)
    (same : observe a = observe b) (different : target a ≠ target b)
    {State : Type r} (initial : Y → State) (advance : Y → State → State)
    (decode : (Nat → State) → Z) :
    ¬ (∀ x, decode (adaptiveTrace initial advance (observe x)) = target x) :=
  no_processing_recovers_erased_distinction observe target a b same different
    (adaptiveTrace initial advance) decode

def SoundAttempt (observe : X → Y) (target : X → Z)
    (attempt : Y → Nat → Option Z) : Prop :=
  ∀ x n value, attempt (observe x) n = some value → value = target x

def CertifiedAbstention (attempt : Y → Nat → Option Z) (reading : Y) : Prop :=
  ∀ n, attempt reading n = none

theorem forced_abstention_at_seam
    (observe : X → Y) (target : X → Z) (a b : X)
    (same : observe a = observe b) (different : target a ≠ target b)
    (attempt : Y → Nat → Option Z) (sound : SoundAttempt observe target attempt) :
    CertifiedAbstention attempt (observe a) := by
  intro n
  cases output : attempt (observe a) n with
  | none => rfl
  | some value =>
    have atA : value = target a := sound a n value output
    have otherOutput : attempt (observe b) n = some value := by rw [← same]; exact output
    have atB : value = target b := sound b n value otherOutput
    exact False.elim (different (atA.symm.trans atB))

theorem both_witnesses_have_certified_abstention
    (observe : X → Y) (target : X → Z) (a b : X)
    (same : observe a = observe b) (different : target a ≠ target b)
    (attempt : Y → Nat → Option Z) (sound : SoundAttempt observe target attempt) :
    CertifiedAbstention attempt (observe a) ∧ CertifiedAbstention attempt (observe b) :=
  ⟨forced_abstention_at_seam observe target a b same different attempt sound,
   forced_abstention_at_seam observe target b a same.symm different.symm attempt sound⟩

theorem no_sound_attempt_is_total
    (observe : X → Y) (target : X → Z) (a b : X)
    (same : observe a = observe b) (different : target a ≠ target b)
    (attempt : Y → Nat → Option Z) (sound : SoundAttempt observe target attempt) :
    ¬ (∀ x, ∃ n value, attempt (observe x) n = some value) := by
  intro total
  obtain ⟨n, value, output⟩ := total a
  have abstains := forced_abstention_at_seam observe target a b same different attempt sound n
  rw [abstains] at output
  cases output
```

```
87
88  theorem adequate_coordinate_must_distinguish
89      (observe : X → Y) (target : X → Z) (a b : X)
90      (same : observe a = observe b) (different : target a ≠ target b)
91      {Coordinate : Type r} (coordinate : X → Coordinate)
92      (adequate : Recovers (fun x => (observe x, coordinate x)) target) :
93      coordinate a ≠ coordinate b := by
94    intro sameCoordinate
95    obtain ⟨decode, correct⟩ := adequate
96    exact two_cases_exclude_every_decoder (fun x => (observe x, coordinate x)) target a b
97      (Prod.ext same sameCoordinate) different decode ⟨correct a, correct b⟩
98
99  theorem essential_coordinate_is_not_recoverable
100     (observe : X → Y) (target : X → Z) (a b : X)
101     (same : observe a = observe b) (different : target a ≠ target b)
102     {Coordinate : Type r} (coordinate : X → Coordinate)
103     (adequate : Recovers (fun x => (observe x, coordinate x)) target) :
104     ¬ Recovers observe coordinate :=
105   universal_class_is_closed observe coordinate a b same
106     (adequate_coordinate_must_distinguish observe target a b same different coordinate adequate)
107
108 theorem recovery_requires_target_constancy
109     (observe : X → Y) (target : X → Z) (adequate : Recovers observe target) :
110     ∀ a b, observe a = observe b → target a = target b := by
111   obtain ⟨decode, correct⟩ := adequate
112   intro a b same
113   exact (correct a).symm.trans ((congrArg decode same).trans (correct b))
114
115 theorem constant_target_is_recovered (observe : X → Y) (value : Z) :
116     Recovers observe (fun _ => value) := ⟨fun _ => value, fun _ => rfl⟩
117
118 theorem identity_readout_recovers (target : X → Z) :
119     Recovers (fun x : X => x) target := ⟨target, fun _ => rfl⟩
120
121 theorem target_retention_recovers (observe : X → Y) (target : X → Z) :
122     Recovers (fun x => (observe x, target x)) target := ⟨Prod.snd, fun _ => rfl⟩
123
124 def bitReadout (_ : Bool) : Unit := ()
125 def bitTarget (x : Bool) : Bool := x
126 def nullAttempt (_ : Unit) (_ : Nat) : Option Bool := none
127
128 theorem bit_seam_excludes_every_decoder : ¬ Recovers bitReadout bitTarget :=
129   universal_class_is_closed bitReadout bitTarget false true rfl (by decide)
130
131 theorem null_attempt_is_sound : SoundAttempt bitReadout bitTarget nullAttempt := by
132   intro _ _ _ impossible
133   cases impossible
134
135 theorem bit_null_has_a_certificate : CertifiedAbstention nullAttempt () :=
136   forced_abstention_at_seam bitReadout bitTarget false true rfl (by decide)
137     nullAttempt null_attempt_is_sound
138
139 theorem bit_distinction_restores_recovery :
140     Recovers (fun x => (bitReadout x, x)) bitTarget := ⟨Prod.snd, fun _ => rfl⟩
141
142 end ClosureConsequences
```

B Complete core axiom audit

The following is the transitive dependency report for every named declaration of the standalone core, including its definitions. The exact signatures and raw JSON messages are supplied in the receipts directory.

```
'UniversalRecoveryClosure.no_processing_recovers_erased_distinction' does not depend on any axioms
'UniversalRecoveryClosure.no_decoder_recovers_erased_distinction' does not depend on any axioms
'ClosureConsequences.Recovers' does not depend on any axioms
'ClosureConsequences.two_cases_exclude_every_decoder' does not depend on any axioms
'ClosureConsequences.universal_class_is_closed' does not depend on any axioms
'ClosureConsequences.adaptiveTrace' does not depend on any axioms
'ClosureConsequences.no_adaptive_transcript_decoder' does not depend on any axioms
'ClosureConsequences.SoundAttempt' does not depend on any axioms
'ClosureConsequences.CertifiedAbstention' does not depend on any axioms
'ClosureConsequences.forced_abstention_at_seam' does not depend on any axioms
'ClosureConsequences.both_witnesses_have_certified_abstention' does not depend on any axioms
'ClosureConsequences.no_sound_attempt_is_total' does not depend on any axioms
'ClosureConsequences.adequate_coordinate_must_distinguish' does not depend on any axioms
'ClosureConsequences.essential_coordinate_is_not_recoverable' does not depend on any axioms
'ClosureConsequences.recovery_requires_target_constancy' does not depend on any axioms
'ClosureConsequences.constant_target_is_recovered' does not depend on any axioms
'ClosureConsequences.identity_readout_recovers' does not depend on any axioms
'ClosureConsequences.target_retention_recovers' does not depend on any axioms
'ClosureConsequences.bitReadout' does not depend on any axioms
'ClosureConsequences.bitTarget' does not depend on any axioms
'ClosureConsequences.nullAttempt' does not depend on any axioms
'ClosureConsequences.bit_seam_excludes_every_decoder' does not depend on any axioms
'ClosureConsequences.null_attempt_is_sound' does not depend on any axioms
'ClosureConsequences.bit_null_has_a_certificate' does not depend on any axioms
'ClosureConsequences.bit_distinction_restores_recovery' does not depend on any axioms
```

C Proof source map

Source	Role in the treatment
`core/UniversalRecoveryClosure.lean`	The universal collision and arbitrary-processing proof, retained as a self-contained formal statement.
`core/ClosureConsequences.lean`	Recovery, two-point exclusion, adaptive transcripts, sound attempts, certified abstention, necessary coordinates and the complete Boolean instance.
`SeamClosure.lean`	Original recovery and image definitions; collision, factorization, transport and equal-standard results.
`SeamIntegration.lean`	Required refinement, retained criterion calibration and the strict abstract prefix ladder.
`SeamClosureConditions.lean`	Adequate lifts, exact countermodels, period transport and the observer-inclusive registration rule.
`DiagonalSeamBridge.lean`	Set/function diagonal core; quotation interface; ordinary and signed provability seams.
`CertificateResidueBridge.lean`	Certificate-or-residue settling and the finite-restart center-pattern reduction.
`Rule30WhiteBoundary.lean`	Full white-boundary classification, all-width non-entry and the actual right-prefix seam.
`LeftSeam.lean`	Left inverse, surjection, cone, entry and temporal ladder; local white/black center observations.
`RingComparator.lean`	Actual seven-cell orbit, identity with the loop block and nonfinite ring starts.
`JointBreak.lean`	Two-sided finite-start reduction to exclusion of a center-period word.
`LeftCheck.lean`	The finite Boolean checker and exact conditional all-width bound statement.
`FiniteWindowSeam.lean`	Finite shadows, finite-prefix bijection, pin pairing, all-horizon finite compatibility and support seams.
All supporting application modules	The unchanged complete imported dependency chains, compiled with the corresponding application target.

The application roots are:

- `applications/diagonal_r6/lean/`
- `applications/parallax_r7/lean/`

Their shared module names are kept in separate trees and built in separate paths. This prevents a naming collision from silently replacing one dependency version with another.

D Named continuity record

The saved *adjudication seam* checkpoint is included in `source_packets/`. Its return rule is to preserve established recovery closure in its unchanged scope; identify the source and criterion of a judgment; distinguish an unissued verdict from a refutation; and apply the same receipt standard to proposals and objections. The present treatment supplies the full mathematical statements supporting that record, including the positive certificate of the null case.

References

[1] F. William Lawvere, *Diagonal arguments and cartesian closed categories*, 1969; reprinted with author commentary in Reprints in Theory and Applications of Categories 15 (2006), 1–13. https://tac.mta.ca/tac/reprints/articles/15/tr15.pdf.

[2] Lawrence C. Paulson, *A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets*, Review of Symbolic Logic (2014). https://www.cl.cam.ac.uk/~lp15/papers/Formath/Goedel-logic.pdf.

[3] Lawrence C. Paulson, with contributions from Janis Bailitis, *Gödel's Incompleteness Theorems*, Archive of Formal Proofs. https://isa-afp.org/entries/Incompleteness.html.

[4] Manuel Eberl, *The Transcendence of π*, Archive of Formal Proofs (2018). https://isa-afp.org/entries/Pi_Transcendental.html.

[5] A. M. Turing, *On computable numbers, with an application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society 42 (1936), 230–265. https://www.cs.ox.ac.uk/activities/ieg/e-library/sources/tp2-ie.pdf.